```
import numpy as np
import matplotlib.pyplot as plt

# ========================================
# 1. Objective Function for WSN Energy/Coverage
# ========================================
def wsn_fitness_function(position, num_nodes, area_size, sink_node):
    """
    Evaluates the quality of a WSN deployment/configuration vector.
    Here, 'position' is a 1D array of coordinates [x1, y1, x2, y2, ..., xN, yN
    Objective: Maximize coverage / Minimize total energy consumption & distanc
    """
    coords = position.reshape((num_nodes, 2))

    # Example WSN metric:
    # 1. Minimize average distance from sensor nodes to the Sink Node (reduces
    # 2. Penalize nodes that are clustered too close together (redundant cover

    total_dist_to_sink = 0.0
    for i in range(num_nodes):
        dist_sink = np.linalg.norm(coords[i] - sink_node)
        total_dist_to_sink += dist_sink

    # Penalty for overlapping/too close nodes
    overlap_penalty = 0.0
    min_safe_dist = 5.0
    for i in range(num_nodes):
        for j in range(i + 1, num_nodes):
            d = np.linalg.norm(coords[i] - coords[j])
            if d < min_safe_dist:
                overlap_penalty += (min_safe_dist - d) * 10.0

    # Fitness: lower is better (minimizing energy/cost function)
    fitness = total_dist_to_sink + overlap_penalty
    return fitness


# ========================================
# 2. Grey Wolf Optimizer (GWO) Algorithm
# ========================================
def grey_wolf_optimizer(obj_func, dim, lb, ub, max_iter, pop_size, *args):
    # Initialize Alpha, Beta, and Delta wolves
    alpha_pos = np.zeros(dim)
    alpha_score = float("inf")

    beta_pos = np.zeros(dim)
    beta_score = float("inf")

    delta_pos = np.zeros(dim)
    delta_score = float("inf")

    # Initialize population randomly within lower (lb) and upper (ub) bounds
    # lb and ub can be arrays or scalars
```

```
    positions = np.random.uniform(lb, ub, (pop_size, dim))
    convergence_curve = np.zeros(max_iter)

    print("\n[INFO] Initializing Grey Wolf Optimization for WSN...")

    for l in range(max_iter):
        for i in range(pop_size):
            # Bound check: bring elements back inside [lb, ub]
            positions[i, :] = np.clip(positions[i, :], lb, ub)

            # Calculate fitness for each wolf
            fitness = obj_func(positions[i, :], *args)

            # Update Alpha, Beta, Delta
            if fitness < alpha_score:
                delta_score = beta_score
                delta_pos = np.copy(beta_pos)
                beta_score = alpha_score
                beta_pos = np.copy(alpha_pos)
                alpha_score = fitness
                alpha_pos = np.copy(positions[i, :])
            elif fitness < beta_score and fitness != alpha_score:
                delta_score = beta_score
                delta_pos = np.copy(beta_pos)
                beta_score = fitness
                beta_pos = np.copy(positions[i, :])
            elif fitness < delta_score and fitness != beta_score and fitness
                delta_score = fitness
                delta_pos = np.copy(positions[i, :])

        # a decreases linearly from 2 to 0 over the course of iterations
        a = 2.0 - l * (2.0 / max_iter)

        # Update the position of search agents (omega wolves)
        for i in range(pop_size):
            for j in range(dim):
                # Alpha influence
                r1, r2 = np.random.rand(), np.random.rand()
                A1 = 2.0 * a * r1 - a
                C1 = 2.0 * r2
                D_alpha = abs(C1 * alpha_pos[j] - positions[i, j])
                X1 = alpha_pos[j] - A1 * D_alpha

                # Beta influence
                r1, r2 = np.random.rand(), np.random.rand()
                A2 = 2.0 * a * r1 - a
                C2 = 2.0 * r2
                D_beta = abs(C2 * beta_pos[j] - positions[i, j])
                X2 = beta_pos[j] - A2 * D_beta

                # Delta influence
                r1, r2 = np.random.rand(), np.random.rand()
                A3 = 2.0 * a * r1 - a
                C3 = 2.0 * r2
                D_delta = abs(C3 * delta_pos[j] - positions[i, j])
```

```
                X3 = delta_pos[j] - A3 * D_delta

                # Average the positions from Alpha, Beta, and Delta
                positions[i, j] = (X1 + X2 + X3) / 3.0

        convergence_curve[l] = alpha_score
        if (l + 1) % max(1, (max_iter // 10)) == 0 or l == 0:
            print(f"Iteration {l + 1}/{max_iter} - Best Fitness (Energy Cost)

    return alpha_score, alpha_pos, convergence_curve


# ========================================
# 3. User Input & Main Execution Flow
# ========================================
if __name__ == "__main__":
    print("=== Wireless Sensor Network (WSN) Energy Optimization using GWO ===
    try:
        num_nodes = int(input("Enter number of sensor nodes (e.g., 10): ") or
        area_size = float(input("Enter square monitoring area size in meters (
        pop_size = int(input("Enter Grey Wolf population size (e.g., 20): ") (
        max_iter = int(input("Enter maximum number of iterations (e.g., 50): "
    except ValueError:
        print("Invalid input. Using default values: 10 nodes, 100m area, 20 wo
        num_nodes, area_size, pop_size, max_iter = 10, 100.0, 20, 50

    # Dimension: each node has (x, y) coordinates
    dim = num_nodes * 2
    lb = 0.0
    ub = area_size

    # Define Sink Node position (e.g., center of the area)
    sink_node = np.array([area_size / 2.0, area_size / 2.0])

    # Run GWO
    best_score, best_pos, conv_curve = grey_wolf_optimizer(
        wsn_fitness_function, dim, lb, ub, max_iter, pop_size, num_nodes, area
    )

    print("\n--- Optimization Results ---")
    print(f"Best Energy/Cost Score: {best_score:.4f}")
    best_coords = best_pos.reshape((num_nodes, 2))
    print("\nOptimal Sensor Node Coordinates (x, y):")
    for idx, coord in enumerate(best_coords):
        print(f"  Node {idx+1}: ({coord[0]:.2f}, {coord[1]:.2f})")

    # Plot Convergence Curve
    plt.figure(figsize=(8, 4))
    plt.plot(conv_curve, color='b', linewidth=2, label='GWO Convergence')
    plt.title('Grey Wolf Optimizer - WSN Energy Optimization Convergence')
    plt.xlabel('Iteration')
    plt.ylabel('Fitness Value (Cost)')
    plt.grid(True)
    plt.legend()
    plt.show()
```

```
# Plot Final WSN Topology
plt.figure(figsize=(6, 6))
plt.scatter(best_coords[:, 0], best_coords[:, 1], c='g', s=80, label='Sens
plt.scatter(sink_node[0], sink_node[1], c='r', marker='s', s=120, label='S
for i, coord in enumerate(best_coords):
    plt.text(coord[0] + 1, coord[1] + 1, f'N{i+1}', fontsize=9)
plt.xlim(0, area_size)
plt.ylim(0, area_size)
plt.title(f'Optimized WSN Topology ({num_nodes} Nodes)')
plt.xlabel('X (meters)')
plt.ylabel('Y (meters)')
plt.grid(True)
plt.legend()
plt.show()
```